

```
import os
import zipfile
import urllib.request
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import reuters
from tensorflow.keras import layers, Sequential
print("TensorFlow version:", tf.__version__)
```

TensorFlow version: 2.20.0

```
# Step 2: Load the 20 Newsgroups dataset
(train_data, train_labels), (test_data, test_labels) = reuters.load_data(
    num_words=10000
)

print("Training samples:", len(train_data))
print("Test samples:", len(test_data))
print("Classes:", len(set(train_labels)))
```

Training samples: 8982
Test samples: 2246
Classes: 46

```
# Step 3: Prepare integer sequences

MAX_LEN = 200

from tensorflow.keras.preprocessing.sequence import pad_sequences

X_train = pad_sequences(
    train_data,
    maxlen=MAX_LEN
)

X_test = pad_sequences(
    test_data,
    maxlen=MAX_LEN
)

y_train = np.array(train_labels)
y_test = np.array(test_labels)

print("X_train shape:", X_train.shape)
print("X_test shape :", X_test.shape)
```

X_train shape: (8982, 200)
X_test shape : (2246, 200)

```
# Step 4: Download and load pre-trained GloVe embeddings

GLOVE_ZIP = "/content/glove.6B.zip"
```

```

GLOVE_TXT = "/content/glove.6B.100d.txt"

# Download GloVe
if not os.path.exists(GLOVE_TXT):
    if not os.path.exists(GLOVE_ZIP):
        print("Downloading GloVe embeddings...")
        urllib.request.urlretrieve(
            "https://nlp.stanford.edu/data/glove.6B.zip",
            GLOVE_ZIP
        )

        with zipfile.ZipFile(GLOVE_ZIP, "r") as zip_ref:
            zip_ref.extract("glove.6B.100d.txt", "/content")

# Load GloVe vectors
embedding_index = {}

with open(GLOVE_TXT, encoding="utf-8") as f:
    for line in f:
        values = line.rstrip().split()
        word = values[0]
        vector = np.asarray(values[1:], dtype="float32")

        if len(vector) == 100:
            embedding_index[word] = vector

print("GloVe words loaded:", len(embedding_index))

# Reuters vocabulary
word_index = reuters.get_word_index()

NUM_WORDS = 10000
EMBEDDING_DIM = 100

# Create embedding matrix
embedding_matrix = np.zeros((NUM_WORDS, EMBEDDING_DIM))

for word, index in word_index.items():
    reuters_index = index + 3

    if reuters_index < NUM_WORDS:
        embedding_vector = embedding_index.get(word)

        if embedding_vector is not None:
            embedding_matrix[reuters_index] = embedding_vector

print("Embedding matrix shape:", embedding_matrix.shape)

```

```

GloVe words loaded: 400000
Embedding matrix shape: (10000, 100)

```

```

# Step 5: Build and compile the text-classification model

model_text = Sequential([
    layers.Input(shape=(MAX_LEN,)),

    layers.Embedding(
        input_dim=NUM_WORDS,

```

```

        output_dim=EMBEDDING_DIM,
        weights=[embedding_matrix],
        trainable=False,
        name="pretrained_glove"
    ),

    layers.GlobalAveragePooling1D(),

    layers.Dense(
        46,
        activation="softmax",
        name="softmax_output"
    )
])

model_text.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model_text.summary()

```

Model: "sequential_2"

Layer (type)	Output Shape	Param
pretrained_glove (Embedding)	(None, 200, 100)	1,000,000
global_average_pooling1d_3 (GlobalAveragePooling1D)	(None, 100)	
softmax_output (Dense)	(None, 46)	4,646

Total params: 1,004,646 (3.83 MB)
 Trainable params: 4,646 (18.15 KB)
 Non-trainable params: 1,000,000 (3.81 MB)

```

# Step 6: Train the model
history_text = model_text.fit(
    X_train,
    y_train,
    epochs=8,
    batch_size=64,
    validation_split=0.1,
    verbose=1
)

```

```

Epoch 1/8
127/127 ————— 3s 14ms/step - accuracy: 0.2897 - loss: 3.00
Epoch 2/8
127/127 ————— 2s 8ms/step - accuracy: 0.4779 - loss: 2.00
Epoch 3/8
127/127 ————— 1s 5ms/step - accuracy: 0.4861 - loss: 2.00

```

```
Epoch 4/8
127/127 ————— 1s 5ms/step - accuracy: 0.4908 - loss: 2
Epoch 5/8
127/127 ————— 1s 5ms/step - accuracy: 0.4996 - loss: 2
Epoch 6/8
127/127 ————— 1s 5ms/step - accuracy: 0.5148 - loss: 2
Epoch 7/8
127/127 ————— 1s 5ms/step - accuracy: 0.5328 - loss: 2
Epoch 8/8
127/127 ————— 1s 5ms/step - accuracy: 0.5455 - loss: 2
```

```
# Step 7: Evaluate the model
test_loss, test_accuracy = model_text.evaluate(X_test, y_test, verbose=0)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
```

```
Test Loss: 2.1297
Test Accuracy: 0.5481
```

```
# Step 8: Plot training and validation accuracy
plt.figure(figsize=(8, 4))
plt.plot(history_text.history["accuracy"], label="Train Accuracy")
plt.plot(history_text.history["val_accuracy"], label="Validation Accuracy")
plt.title("Text Classification Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```



